

Relatório Técnico - Projeto SISOP2

Daniel Gutschwager e Pedro Mews

¹Curso de Ciência da Computação
Universidade Federal do Rio Grande do Sul (UFRGS)
Porto Alegre, RS – Brasil

1. Introdução

Este relatório descreve a implementação de um sistema distribuído de transferência de valores entre clientes, simulando um sistema de pagamento similar ao PIX. O projeto foi desenvolvido como parte da disciplina de Sistemas Operacionais II, utilizando C++ e a API UDP para comunicação inter-processos.

O sistema é composto por um servidor central e múltiplos clientes, permitindo a realização de transações financeiras de forma concorrente e distribuída. A implementação foca em garantir a correta atualização dos saldos dos clientes, a consistência do saldo total do banco e a manutenção de um histórico preciso das transações.

2. Implementação dos Subserviços

2.1. Subserviço de Descoberta

O subserviço de descoberta é implementado tanto no cliente quanto no servidor:

- **Cliente:** Na classe `Client`, o método `start_discovery()` inicia uma thread que envia pacotes de descoberta em broadcast. Ao receber uma resposta do servidor, armazena o endereço e finaliza o processo.
- **Servidor:** Implementado na função `discovery_server_loop()` em `discovery.cpp`. Esta função escuta por pacotes de descoberta, registra novos clientes e envia respostas de confirmação.

2.2. Subserviço de Processamento

O processamento de transações é gerenciado principalmente pela classe `Server` e `ServerState`:

- `Server::handle_request()`: Processa requisições individuais, verificando saldos, atualizando a tabela de clientes e enviando respostas.
- `ServerState::process_req()`: Realiza a lógica de processamento da transação, incluindo verificações de sequência, saldo suficiente e atualização dos saldos.

2.3. Subserviço de Interface

A interface do usuário é gerenciada por threads dedicadas tanto no cliente quanto no servidor:

- **Cliente:** `Client::start_interface()` gerencia uma fila de mensagens para exibição.
- **Servidor:** `Server::start_interface()` processa uma fila de informações para exibição, incluindo detalhes das transações e estatísticas do sistema.

3. Áreas de Sincronização

As principais áreas que requerem sincronização incluem:

- **Acesso à tabela de clientes:** Utiliza-se um `std::shared_mutex` em `ServerState` para permitir múltiplas leituras simultâneas, mas escrita exclusiva.
- **Atualização de estatísticas:** Um `std::mutex` protege as atualizações das estatísticas globais do servidor.
- **Filas de mensagens:** Tanto no cliente quanto no servidor, mutexes e variáveis de condição são usados para sincronizar o acesso às filas de mensagens para a interface.

4. Principais Estruturas e Funções

- `packet_t`: Estrutura que representa um pacote de comunicação, incluindo tipo, número de sequência e payload.
- `ServerState`: Gerencia o estado global do servidor, incluindo a tabela de clientes e estatísticas.
- `Client`: Implementa a lógica do cliente, incluindo descoberta, processamento de transações e interface.
- `Server`: Gerencia as operações do servidor, incluindo recepção de pacotes e distribuição para processamento.
- `udp_send()` e `udp_receive()`: Funções de utilidade para envio e recebimento de pacotes UDP.

5. Uso de Primitivas de Comunicação Inter-processos

O projeto utiliza exclusivamente sockets UDP para comunicação:

- `socket()`: Criação de sockets UDP.
- `bind()`: Associação de sockets a portas específicas.
- `sendto()` e `recvfrom()`: Envio e recebimento de datagramas UDP.
- `select()`: Utilizado para implementar timeouts na recepção de pacotes.

A serialização e desserialização de pacotes são realizadas manualmente, convertendo entre a representação em memória e a representação de rede.

6. Problemas Encontrados e Soluções

6.1. Não Previsibilidade do Cenário DUP

Um dos desafios encontrados foi durante os testes de consistência para o caso de cobertura DUP, em que a requisição é enviada e processada corretamente, porém o sinal de ACK de resposta não chega ao cliente, que por sua vez reenviará a requisição. Esse problema não foi resolvido

6.2. Testes e Depuração em Ambiente Distribuído

Os testes e a depuração do sistema em múltiplos computadores apresentaram desafios significativos devido à natureza distribuída da aplicação.

- **Problema:** Dificuldade em reproduzir cenários de teste consistentes em diferentes máquinas e ambientes de rede.
- **Solução:** Desenvolvimento de um script de teste de estresse capaz de simular múltiplos clientes em um ambiente Docker controlado, permitindo executar experimentos de forma mais previsível e repetível.
- **Problema:** Complexidade na correlação de eventos entre diferentes processos e máquinas, o que dificultava a identificação da origem de falhas e comportamentos inesperados.
- **Solução:** Implementação de um sistema de *logging* detalhado e padronizado em todos os componentes, possibilitando a reconstrução precisa da sequência de eventos e facilitando a depuração em ambiente distribuído.

6.3. Inconsistência temporária de estado entre servidor e cliente devido a retransmissões simultâneas

Ocorrência de inconsistências temporárias de estado entre o cliente e o servidor durante retransmissões simultâneas de múltiplos clientes. Em alguns casos, o servidor processava transações na ordem incorreta, gerando divergências momentâneas nos saldos.

- **Solução:** Adoção de bloqueios adicionais no processamento de requisições críticas e verificação de integridade após cada operação, garantindo que o estado final refletisse corretamente todas as transações confirmadas.

7. Conclusão

O projeto implementou com sucesso um sistema distribuído de transferência de valores, atendendo aos requisitos de concorrência, consistência e tolerância a falhas. Os principais desafios encontrados foram relacionados à natureza não confiável do UDP e à complexidade inerente de sistemas distribuídos. As soluções adotadas, como o uso de números de sequência, mecanismos de retry e logging detalhado, permitiram superar esses desafios e criar um sistema robusto e funcional.